

Integrating Sample Inheritance into Bayesian Optimization for Evolutionary Robotics

K. Ege de Bruin¹, Kyrre Glette^{1,2}, and Kai Olav Ellefsen¹

¹Department of Informatics, University of Oslo, Norway

²RITMO, University of Oslo, Norway
egedebruin@gmail.com

Abstract

In evolutionary robotics, robot morphologies are designed automatically using evolutionary algorithms. This creates a body-brain optimization problem, where both morphology and control must be optimized together. A common approach is to include controller optimization for each morphology, but starting from scratch for every new body may require a high controller learning budget. We address this by using Bayesian optimization for controller optimization, exploiting its sample efficiency and strong exploration capabilities, and using sample inheritance as a form of Lamarckian inheritance. Under a deliberately low controller learning budget for each morphology, we investigate two types of sample inheritance: (1) transferring all the parent’s samples to the offspring to be used as prior without evaluating them, and (2) reevaluating the parent’s best samples on the offspring. Both are compared to a baseline without inheritance. Our results show that reevaluation performs best, with prior-based inheritance also outperforming no inheritance. Analysis reveals that while the learning budget is too low for a single morphology, generational inheritance compensates for this by accumulating learned adaptations across generations. Furthermore, inheritance mainly benefits offspring morphologies that are similar to their parents. Finally, we demonstrate the critical role of the environment, with more challenging environments resulting in more stable walking gaits. Our findings highlight that inheritance mechanisms can boost performance in evolutionary robotics without needing large learning budgets, offering an efficient path toward more capable robot design.

Data/Code available at: <https://tinyurl.com/alife2025-egedebruin>

Introduction

Traditionally, robots are designed manually by engineers to perform specific tasks. While methods like reinforcement learning are often used to optimize control systems, the morphology of most modern robots is still largely human-designed. This approach can result in efficient robots for predefined tasks. However, it limits the robot’s ability to adapt to new tasks or environments, as its morphology is not optimized for adaptability beyond its initial design.

In nature, we see many organisms that can quickly adapt to new environments and perform various tasks. This is

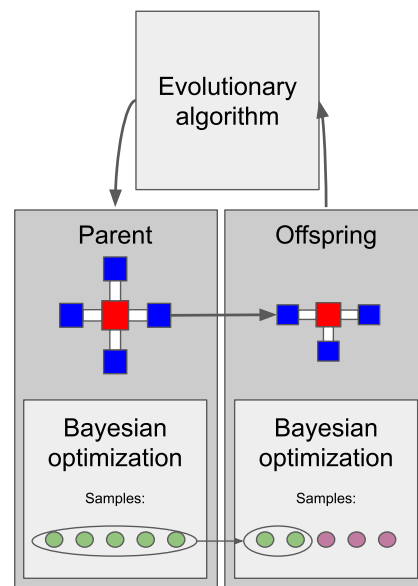


Figure 1: Overview of an evolutionary loop for morphology optimization with Bayesian optimization-driven controller learning, and controller inheritance methods.

the inspiration for evolutionary robotics, where robots are being designed using evolutionary algorithms (Lipson and Pollack, 2000; Nolfi et al., 2016; Sims, 1994). A somewhat unique problem for robots is the co-optimisation of control and morphology (Cheney et al., 2016; Faña et al., 2013). The evolutionary algorithm needs to traverse a more complex search space due to the intertwined body and brain. Consequently, there might be a mismatch between control and robot morphology, and a robot’s morphology might be discarded because it has not been performing to its full potential (Eiben et al., 2013). This causes the algorithm to stagnate quickly, without exploring much of the design space, because new, potentially well-performing robot morphologies are being discarded.

A common approach to this problem is to make a robot adapt its control to its new morphology, increasing the

chance of performing to its morphology’s potential. This is often done by adding an inner learning loop to the evolutionary algorithm: An outer evolutionary loop searches through the morphology search space. In contrast, an inner learning loop tries to find optimal control parameters for every new robot. Adding a learning loop is an efficient way to find well-performing robots (Gupta et al., 2021; Luo et al., 2022; Miras et al., 2020; Zhao et al., 2020).

If every new robot morphology goes through a controller-learning phase, one could follow a Darwinian approach, where control parameters are either randomly initialized for every robot morphology (Gupta et al., 2021), or offspring robots inherit *initial* control parameters from parents (Miras et al., 2020). It is also possible to follow a Lamarckian approach, where offspring robots inherit the *learned* controllers of their parents (Harada and Iba, 2024; Jelisivcic et al., 2019; Luo et al., 2023). This way, information learned by parents can be transferred to offspring robots to kickstart the learning process.

A common problem in using Lamarckian inheritance is that offspring robots’ morphologies are different from each other, where the number and location of sensors and actuators can differ. This raises the challenge of which information to transfer from parent to offspring. Jelisivcic et al. (2019) solve the problem by using a neural network as a genotype for both the evolutionary and learning processes, while other work keeps an archive of similar robots to inherit from (Le Goff et al., 2022), saves information for all possible locations in which an actuator can be located at (Luo et al., 2023) or only transfers weights that are shared among neural network controllers (Harada and Iba, 2024).

Bayesian optimization is a learning algorithm with a fast learning capability and sample efficiency (Chatzilygeroudis et al., 2019; Lan et al., 2021; Le Goff et al., 2022; van Diggelen et al., 2024). In this paper, we investigate methods to inherit information between generations in an evolutionary robotics setting using Bayesian optimization. Because we have to transfer information in a Bayesian optimization setting, we require different approaches than those used in other learning settings. Our main contributions are the following:

- New techniques for inheriting control parameters in a Bayesian optimization controller learning setting.
- Analysis of how the advantage of inherited control is affected by the similarity between parent and offspring, and how it depends on the robot’s environment.

To our knowledge, this is the first Lamarckian Inheritance method developed for controllers learned with Bayesian Optimization. This allows the sample-efficient adaptations from BO to be accumulated across generations of evolution.

Related work

Body-brain optimization

In the introduction, we discussed the problem of body-brain optimization, where a mismatch between robot morphology and control can cause stagnation and lack of diversity (Eiben et al., 2013). A common solution to this problem is to add controller optimization for every robot morphology, often called a learning loop (Gupta et al., 2021; Luo et al., 2022; Miras et al., 2020; Zhao et al., 2020). Learning usually requires a high learning budget to give the robot enough time to adapt to the morphology. In our work, we limit this learning budget using Bayesian optimization. Other approaches include *innovation protection* (Cheney et al., 2018). It is similar to controller optimization in that robot morphologies are evaluated on multiple controller parameters, but the focus is more on adding this to innovative morphologies. Another approach is to train a single controller on a diverse set of morphologies, to be used as a teacher for new robot morphologies (Mertan and Cheney, 2024). Finally, to deal with a lack of diversity *quality-diversity* algorithms such as novelty search with local competition (Lehman and Stanley, 2011) and MAP-Elites (Mouret and Clune, 2015; Nordmoen et al., 2021) have been used to enforce both the quality and diversity of solutions. These approaches need an explicit diversity measure to steer the algorithm towards more diverse solutions, and choosing this measure poses another challenge in the design of the algorithm (Norstein et al., 2022).

Bayesian optimization

For optimizing robot controllers, we apply Bayesian optimization (Mockus, 1974), which is known for its sample efficiency, and its ability to balance the exploration-exploitation trade-off. It uses a surrogate function to predict the objective value using the parameters to optimize, such as robot-control parameters in a robotics setting. Gaussian processes (Williams and Rasmussen, 2006) are commonly used as surrogate functions because they naturally handle uncertainty and can be customized to fit the problem setting (Saito and Oka, 2024). This makes the algorithm computationally expensive for high-dimensional problems with many samples, which is why other techniques are usually preferred for morphology optimization (Bhatia et al., 2021). However, with a limited dimensionality and sample size it is a good method for controller learning (Lan et al., 2021; van Diggelen et al., 2024), where priors can also be exploited to kickstart the optimization process (Chatzilygeroudis et al., 2019; Cully et al., 2015).

Lamarckian evolution

During the process of evolving robots, when new robots are created they often inherit control parameters from their parent robots. One could follow a Darwinian approach, where control parameters are either randomly initialized for every robot morphology (Gupta et al., 2021), or offspring robots

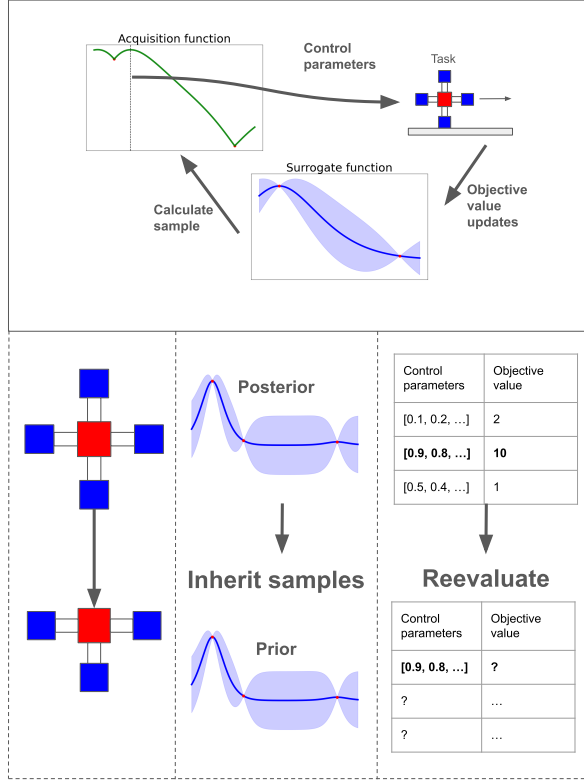


Figure 2: Overview of Bayesian optimization and the inheritance methods. Bayesian optimization iteratively samples the control parameters with the highest acquisition value, which considers the surrogate function’s mean and uncertainty. Inheritance is done using two methods. With *inherit samples*, all samples with their measured performance are transferred without evaluating them, this is the same as transferring the parent’s posterior to be the offspring’s prior. With *reevaluate*, the best-performing samples from the parent are reevaluated by the offspring, leaving the rest of the learning budget for the Bayesian optimization process.

inherit *initial* (pre-learning) control parameters from parents (Miras et al., 2020). Another approach is to use Lamarckian evolution, where offspring robots inherit *learned* control parameters from parents. Lamarckian evolution has received limited attention in evolutionary robotics, but studies have demonstrated its advantages over Darwinian evolution. Jelisavcic et al. (2019) showed that inheriting learned controllers, rather than initial populations, improves performance. Similarly, Luo et al. (2023) found that passing learned parameters to offspring enhances adaptation. Le Goff et al. (2022) explored a related approach, using an archive to ensure inherited parameters remained compatible across different robot configurations. Lastly, Harada and Iba (2024) use a transfer learning approach where neural network weights are shared, and they show an improvement

when using crossover as well. All these works have shown that bootstrapping the learning process of robot morphologies using learned information from previous generations can improve the whole evolutionary process. In our work, we apply Lamarckian inheritance in a Bayesian optimization setting, showing the generalizability of Lamarckian inheritance in evolvable robots. We use a different form of Lamarckian inheritance closer to social learning, which allows an agent to learn by combining its experiences with experiences from other agents (Bartoli et al., 2020). Other works transfer control parameters directly, while we transfer samples of the Bayesian optimization process to either reevaluate them or use them as prior for the next agent.

Sample inheritance

Bayesian optimization samples parameters iteratively in the following manner: First, it uses a surrogate function to predict performance as a function of parameters. Then, it uses an acquisition function to sample the next most promising control parameters. It makes this prediction using the predicted mean and uncertainty for an optimal exploration-exploitation trade-off. The parameters are then evaluated, and the resulting objective value is used to update the surrogate. In our evolutionary robotics setting, the objective value is predicted as a function of the *control parameters*. This prediction is based on previous samples. Therefore, with more samples, the prediction becomes more accurate.

In an evolutionary robotics setting, the controller optimization process can be initialized from scratch for every new morphology, and this is what others (de Bruin et al., 2025; Gupta et al., 2021; Luo et al., 2022; Miras et al., 2020; Zhao et al., 2020) have done in previous work. A common reason for starting from scratch is that there is a need for a mapping between parent and offspring to transfer information. This mapping can limit the choice of controllers. However, starting from scratch loses potential information that could be transferred from the parent robot to the offspring robot. This work investigates methods to transfer experience over generations in a setting with Bayesian optimization as controller optimization. We do this by transferring samples from parent robots to offspring robots. The sample inheritance approaches we try in this work are the following:

1. **Inherit samples:** Offspring robots inherit samples from their parents, and use them as already sampled points, while not reevaluating them.
2. **Reevaluate:** Offspring robots reevaluate the best samples from their parents.

After inheritance, all robots continue sampling further using Bayesian optimization in both cases. In the *inherit samples* approach, the offspring inherits all samples, including the measured performance of the parent robot, without reevaluating them. The upside of this approach is that more

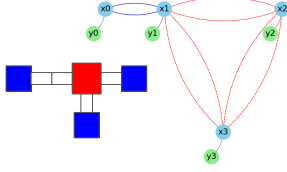


Figure 3: An example robot with its CPG network. The white modules are the actuators. A blue solid line represents a Manhattan distance of 1, and a red dotted line represents a Manhattan distance of 2.

information is being transferred, even though the information is less accurate because it is not being evaluated on the same robot (since the parent likely had a different morphology than the offspring). In the *reevaluate* approach, the offspring reevaluates the best samples from the parent. The upside of this approach is that the samples’ measured performance will be more accurate because the control parameters are evaluated on the actual offspring robot. We ensure that the learning budget for both approaches is the same, meaning that when reevaluating the samples in the *reevaluate* approach, these evaluations are taken from the offspring’s budget. An overview of these approaches can be seen in Figure 2.

Method

Robot phenotype

For this work, we make use of Revolve2¹ as a modular robot framework, the modules are based on the RoboGen framework (Auerbach et al., 2014). These modules are designed in the MuJoCo simulator, and it is possible to 3D print and attach them, to evaluate the robots in real life. The morphologies are built by attaching building blocks using three different modules: A head module, a block module, and a joint module. There is no restriction on the number of block and joint modules, but every robot has exactly one head module. The head module has four attachment points, the block has six, and the joint has two. Figure 4 shows how the three modules look.

The joint module is the only movable module. We use a decentralized controller and use two controller mechanisms to actuate the joints. Both mechanisms offer a simple and effective approach to generating rhythmic patterns for locomotion. The first is a sine-wave controller using the following equations:

$$\begin{aligned}\Theta &= A * \sin(\phi + P) + O \\ \phi &= \phi + \Delta_\phi * F\end{aligned}\quad (1)$$

In these equations, Θ is the output for the controller, A is the amplitude, P is the phase offset, O is the angular offset,

¹See <https://github.com/ci-group/revolve2>

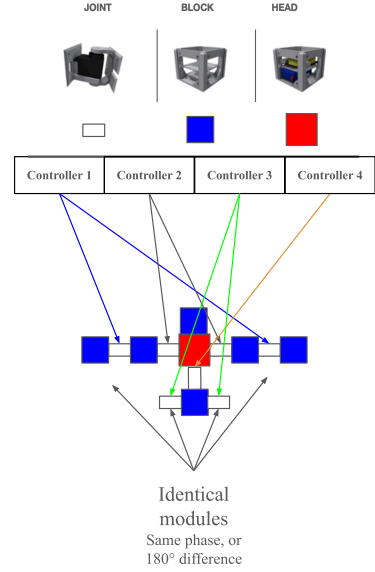


Figure 4: There are three types of modules, and every robot always has one head module. The head module has four attachment points, the block module has six (it includes up and down) and the joint module has two. Symmetry is ensured by making the left and right parts of the central axis identical. Every joint module has its own sine wave parameters, but the symmetrical parts share parameters.

and F is the frequency. A , P and O are the three learnable parameters, and F is set to 4.

The second controller mechanism is a Central Pattern Generator (CPG) controller. CPGs are neural networks responsible for animals’ rhythmic movement, without sensory information or rhythmic input (Ijspeert, 2008). The idea of the CPG network used in our network is to have neurons that work as differential oscillators, creating more complex oscillating patterns than a simple sine wave. In our work, each joint i has two neurons, x_i and y_i , that are connected using the following equations, where the x_i neuron is responsible for the actuator’s output. Note that N_i is the set of neighboring joints of joint i , a neighbor is any joint within a Manhattan distance of 2.

$$\Delta x_i = w_i y_i + \sum_{j \in N_i} w_{ji} x_j \quad (2)$$

$$\Delta y_i = -w_i x_i \quad (3)$$

The initial state of each neuron is $\frac{1}{2}\sqrt{2}$. We define three learnable parameters for each joint: w_i for the internal connection, w_{ji_1} for each neighbouring joint with a Manhattan distance of 1, and w_{ji_2} for each neighbouring joint with a Manhattan distance of 2. Figure 3 shows an example robot with its CPG network.

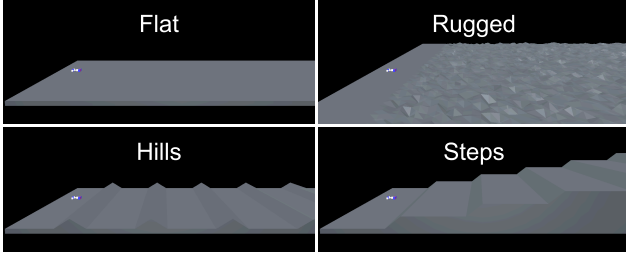


Figure 5: The four environments used in this work.

Even though the control is decentralized for both the sine-wave and the CPG controller, joints may share parameters due to symmetry. This will be explained further in the next subsection. An overview of the robot phenotype can be seen in Figure 4.

Robot genotype

The robot morphology is directly encoded in the genotype and can be represented as a tree. Every robot has a head module as the root of the tree. We ensure symmetry by making the left and right sides of the robot identical. Therefore, there are three attachment points to the head module in the genotype, one for the front, one for the back, and one for the left and right sides of the head module. This is the same for all the block modules in the center of the robot. The head module is always in the centre of the robot, and every module placed in front or at the back of the core module is also labelled as a center module. Every joint module in the genotype has a sine wave as a controller. Due to symmetry, a joint on the left side of the centre has the same sine-wave parameters as its equivalent on the right side. We enforce symmetry to more closely resemble real-life creatures and to reduce the search space.

There are three possible *body mutations*: Firstly, a random number of modules can be added to the robot. Secondly, a random number of modules can be removed from the robot. Lastly, there is a parameter that decides whether the symmetrical parts of the robot are in the same phase or in an alternating phase, and the third mutation switches this. We add the possibility of an alternating phase between the symmetrical parts to enforce a walking-like pattern. The third mutation only affects the sine-wave controller. For the CPG controller there is no specific parameter that can alternate the phase between symmetrical parts, which is why for the CPG controller this mutation has no effect.

In the previous section, it was shown that both the sine-wave and CPG controller have three parameters per joint. When a new joint is created, a new controller is made with random parameters, and when one is removed, this controller is also removed. For the evolutionary search of control parameters, we use a *controller mutation* that changes these parameters by adding or subtracting Gaussian noise

Method	Inherited	Reevaluated	Fresh
Inherit Samples	30	0	30
Reevaluate	5	5	25

Table 1: Number of samples **Inherited** from the parent, which of them are **Reevaluated**, and **Fresh** chosen by the Bayesian optimization process

from these parameters.

Controller optimization

The control learning algorithm is Bayesian optimization, which was chosen due to its fast learning capability and sample efficiency. Bayesian optimization has a high computational complexity, but due to our low number of controller parameters and low learning budget, this issue is minimal for our experiments. We use the Matern 5/2 kernel with a length scale of 0.2 as the surrogate function, and the Upper Confidence Bound as the acquisition function with an exploration variable of 3, which has been used and worked well before in evolving robots for directed locomotion (Lan et al., 2021; van Diggelen et al., 2024). Samples evaluated by the robot have an uncertainty of 0. For the *Inherit samples* experiments, we add an uncertainty of 2 to the inherited samples because this was the best-performing value in preliminary experiments. We use a learning budget of 30 de Bruin et al. (2025), and in the *reevaluate* setting, 5 samples are transferred from the parent robot to the offspring. This means that, with *reevaluate*, offspring robots repeat the 5 best samples from their parents, leaving 25 samples for the Bayesian optimization process. In the *inherit samples* setting, all 30 samples are transferred to the offspring robot without being evaluated, but only to be used as prior.

As explained above, inheriting controller samples is made challenging by the fact that the offspring may not have the same joints to control as its parent had. The direct encoding used in this work makes it possible to track which control parameters can be inherited. Two mutations can change the robot morphology: If a joint module is added, its control parameters are set to random, and if a joint module is removed, its control parameters are removed as well from the sample inheritance. The control parameters of the joints that have not been added or removed are transferred directly from the parent to the offspring robots.

Morphology optimization

An evolutionary algorithm searches the robot morphology search space to find the best morphology for the task. The population size is 200, and every new robot morphology goes through a learning phase to find optimal control parameters. In each generation, 20 new robots are created by selecting parents through tournament selection with a tournament size of 5. The offspring replace the oldest robots

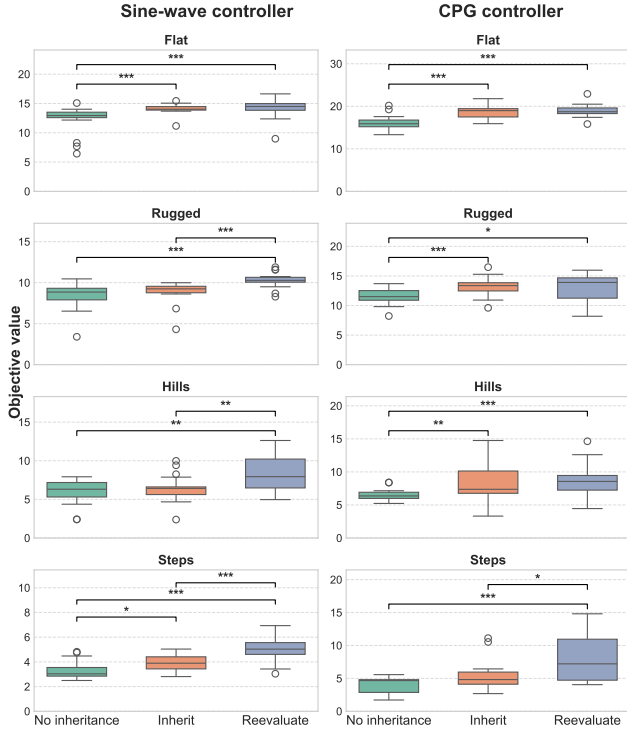


Figure 6: Boxplot of the best-performing robots for all 20 runs. We show significant differences where * is $p < 0.05$, ** is $p < 0.01$ and *** is $p < 0.001$.

in the population to ensure performance and diversity. New robots are created using asexual reproduction. We perform 500 generations, equalling around 300,000 function evaluations per run. Robots in the initial population have a size of between 15 and 20 modules, the add and remove mutation adds or removes up to 3 modules, and the maximum size is 20. If the robot size becomes too large, another mutation is tried. The size limit is chosen not to make the robots too large, as it should be possible to make them in real life. For the evolutionary search of the control parameters, Gaussian noise is added to the parameters with a mean of 0 and a standard deviation of 0.1.

There are four environments: Flat, hilly, rugged, and steps; see Figure 5 for an example robot in the four environments. The task is to move forward as far as possible and the objective value of the robot is the displacement from beginning to end in 30 seconds. We compare the two sample-transferring approaches to a baseline where there is no sample transfer. In that case, the initial control parameters per morphology come from an evolutionary search with the mutation explained in the genotype section. For all methods, the morphology optimization is the same evolutionary algorithm as described. We perform 20 runs for each parameter configuration and perform Mann-Whitney U tests to test significant differences.

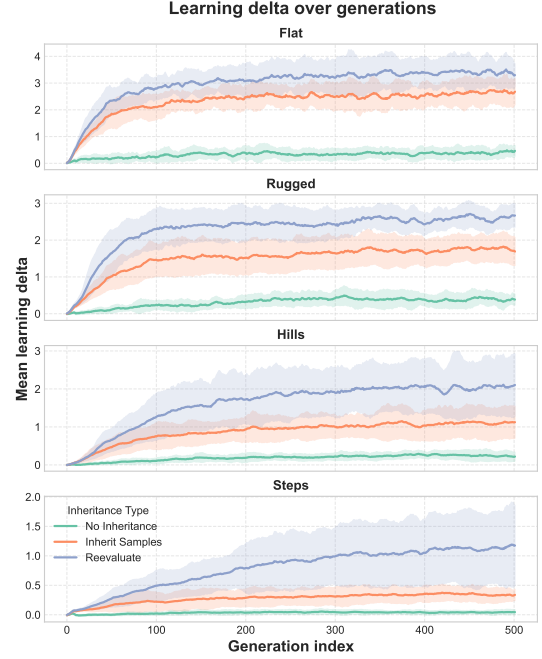


Figure 7: The learning delta is plotted over generations. We estimate the learning delta by calculating the advantage gained from the Bayesian optimization process. It is the robot's performance after learning compared to the robot with random control. We only show the learning delta for the sine-wave controller.

Results

Performance

Figure 6 shows the performance of the inheritance mechanisms with the sine-wave and CPG controllers, in the 4 different environments. For the sine-wave controller, *reevaluate* consistently performs better, except for the *flat* environment, where *inherit samples* has similar performance. *Inherit samples* performs better than *no inheritance* only in the *flat* and *steps* environment, with a more significant difference in the *flat* environment. For the CPG controller, both *reevaluate* and *inherit samples* perform better than *no inheritance*, except for the *steps* environment, where only *reevaluate* has significantly better performance. Due to the limiting frequency of 4 for the sine-wave controller, we do not compare the two controllers.

Figure 7 shows the *learning delta* over generations, which estimates how much the robot gains from Bayesian optimization, compared to a robot with a random controller. To avoid unfair disadvantages for random controllers, their performance is found by taking the best of 30 random samples on the given morphology. The learners spend these 30 sam-

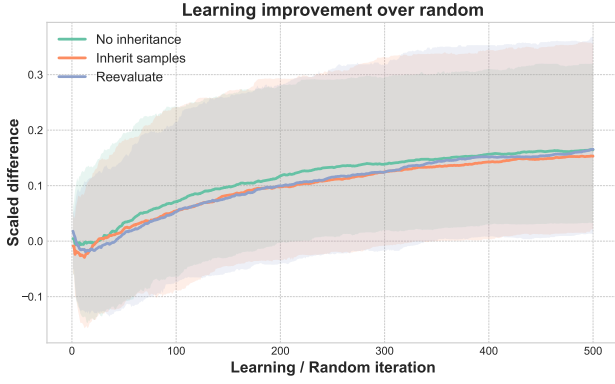


Figure 8: Average scaled improvement of learned over random controllers across iterations. Shaded areas show an interquartile range (25th to 75th percentile) to indicate variability. The analysis includes 20 randomly sampled robots per run, totalling 4,800 robots. All learning started from scratch; the colors of the lines show from which type of run the robots were taken.

ples searching for control parameters with Bayesian optimization, as normal. The results show that learning without Lamarckian inheritance (*no inheritance*) does not perform much better than random sampling. However, adding inheritance does lead to an increase in the learning delta over generations. The main reason that *no inheritance* does not improve much on a random search is likely due to the limited learning budget, and Figure 8 supports this claim. We sampled a total of 4,800 robots from all runs and compared learning to random over a larger learning budget. The plot shows that with a larger learning budget, Bayesian optimization performs better than random - even without inheritance.

Difference to parent

Not every mutation to a robot’s morphology is successful, but a good learning algorithm should ensure that offspring have more successful mutations. In Figure 9, we present the number of robots for each tree edit distance value, as used in previous work (Samuelsen and Glette, 2014), relative to their parent. We show it for all robots and for the robots that have a higher fitness than their parent. Most robots have a low tree edit distance to their parent, and with a tree edit distance of 1, there are more occurrences of *improving* robots for *reevaluate* and *inherit samples* than *no inheritance*. This is despite that the two inheritance schemes have a fewer *total* number of samples with a tree edit distance of 1 compared to *no inheritance*. Moreover, the number of improved samples for the inheritance schemes is already less than *no inheritance* at a tree edit distance of 2, showing that inheritance is only beneficial for robots that are similar to each other. Finally, more robots are similar to their parents than those that are not similar to their parents. Note that the plot rep-

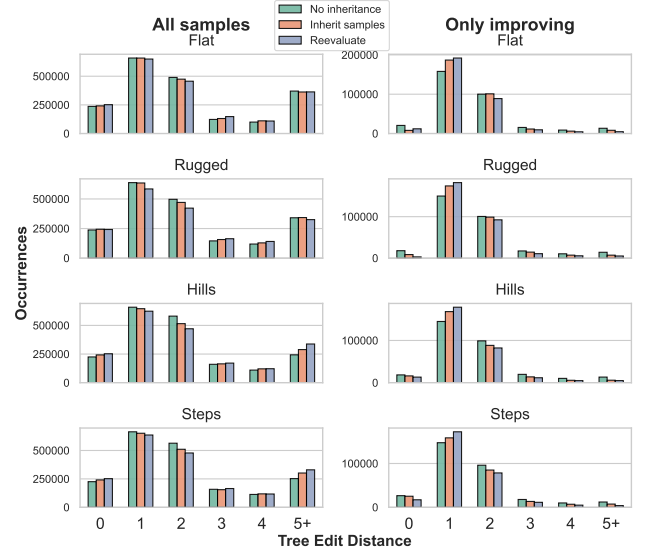


Figure 9: Bar plots showing the number of robots for the sine-wave controller where the tree edit distance is compared to its parent. The left plots show the occurrences for all robots and the right plot for those that have a higher fitness than their parent.

resents the sine wave controller, meaning that a robot with a tree edit distance of 0 from its parent differs in the phase between its symmetrical parts. In this case, the morphology likely requires a different control strategy.

Environment

Figure 10 presents the distribution of locomotion types for the robot that performs the best in each run. We consider swimming and walking the most stable forms of locomotion, while rolling is the least stable. The results demonstrate the influence of the environment on locomotion strategy. Robots predominantly adopt rolling locomotion in both flat and rugged terrains. Environments with hills and steps favour more stable movement patterns, such as swimming and walking, showing more diversity in movement patterns. The main difference between controller mechanisms is the lack of the *worm* locomotion type in the CPG controller. Figure 11 shows typical robots for the four specified movement gaits².

Discussion

Our study of Lamarckian inheritance of robot control parameters using Bayesian optimization as controller optimization has indicated that inheriting samples works well. It works well both when reevaluating samples on the offspring, and when using parents’ sampled values directly without reevaluation. We also show the importance of the environment,

²Videos of example robots can be found on the repository page: <https://tinyurl.com/alife2025-egedebuin>

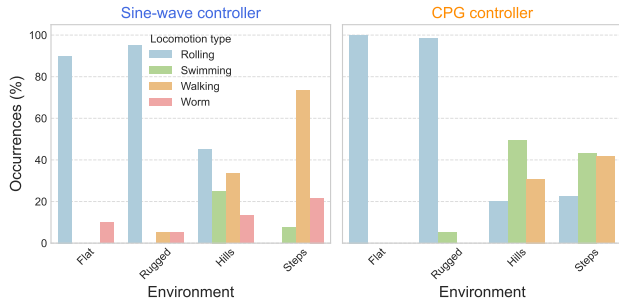


Figure 10: The locomotion type by environment of the best robot in every run. It shows the percentage of the locomotion type for every controller-environment combination. Swimming and walking are considered to be more stable than worming, which in turn is more stable than rolling.

regardless of the inheritance mechanism. In more challenging environments, such as the hills and steps environment, the search resulted in more stable gaits. Finally, we have shown that the method of transferring samples works with two types of controllers and four environments, where in more challenging environments it results in a more diverse set of walking gaits.

We also highlight the advantage of adding inheritance: it can improve performance without requiring a large learning budget. The findings suggest that incorporating inherited solutions into the offspring’s learning process can produce better results, even when the optimization algorithm itself does not perform particularly well or has a too-low budget. Additionally, the use of direct encoding with a low mutation rate supports the idea that similar parent robots lead to offspring with comparable or even improved performance.

It is important to consider how an indirect encoding might affect the results because the mapping from parent to offspring in such an encoding is not straightforward. An indirect encoding could potentially lead to greater diversity in offspring. This highlights an area for future work since exploring how this impacts the optimization process could show useful insights into the influence of Lamarckian inheritance.

Conclusion

We have investigated the influence of information exchange within generations of evolvable robots. To our knowledge, this is the first work to explore the combination of Bayesian optimization as a learning algorithm with Lamarckian inheritance. We transfer samples from parent to offspring in two methods: *inherit samples* uses all parent samples as prior for the offspring, and *reevaluate* uses the best samples from the parent robot and evaluates them on the offspring. We do this on a direct morphology encoding and use two decentralized controller mechanisms: a sine-wave controller and

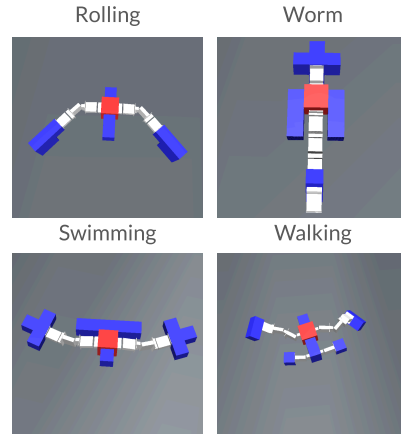


Figure 11: Typical robot morphologies for the four specified gaits.

a two-neuron CPG controller. The results show that *reevaluate* has the best performance and that *inherit samples* often improves on *no inheritance* as well.

Our robots are given a small learning budget, which lowers their benefit from learning in the standard case (no inheritance). However, adding an inheritance scheme greatly increases robots’ *learning delta*, suggesting that Lamarckian inheritance can be particularly valuable for Bayesian Optimization in settings where gathering new data is costly.

All methods work best with robots that are similar to their parents, and this difference is most prevalent for *inherit samples* and *reevaluate*. We therefore showed that our Lamarckian inheritance does not work well with robot bodies that are very different from each other. The evaluation of a robot for a task is the result of the interaction between its body and the environment. Therefore, future work can look into the effect of Lamarckian evolution on changing environments. Moreover, it would also be interesting to look into more complex tasks, environments, or controllers where controller optimization more important.

Acknowledgements

This work was partially supported by the Research Council of Norway through its Centres of Excellence scheme, project number 262762. The experiments were performed on the Fox Cluster, owned by the University of Oslo IT Department.

References

- Auerbach, J., Aydin, D., Maesani, A., Kornatowski, P., Cieslewski, T., Heitz, G., Fernando, P., Loshchilov, I., Daler, L., and Floreano, D. (2014). Robogen: Robot generation through artificial evolution. In *Artificial Life Conference Proceedings, ALIFE*. MIT Press.
- Bartoli, A., Catto, M., De Lorenzo, A., Medvet, E., and Talamini, J. (2020). Mechanisms of social learning in evolved artificial

- life. In *The 2020 Conference on Artificial Life*, ALIFE 2020. MIT Press.
- Bhatia, J., Jackson, H., Tian, Y., Xu, J., and Matusik, W. (2021). Evolution gym: A large-scale benchmark for evolving soft robots. *Advances in Neural Information Processing Systems*, 34:2201–2214.
- Chatzilygeroudis, K., Vassiliades, V., Stulp, F., Calinon, S., and Mouret, J.-B. (2019). A survey on policy search algorithms for learning robot controllers in a handful of trials. *IEEE Transactions on Robotics*, 36(2):328–347.
- Cheney, N., Bongard, J., Sunspiral, V., and Lipson, H. (2016). On the Difficulty of Co-Optimizing Morphology and Control in Evolved Virtual Creatures. *Proceedings of the Artificial Life Conference 2016 (ALIFE XV)*, pages 226–234.
- Cheney, N., Bongard, J., SunSpiral, V., and Lipson, H. (2018). Scalable co-optimization of morphology and control in embodied machines. *Journal of The Royal Society Interface*, 15(143):20170937.
- Cully, A., Clune, J., Tarapore, D., and Mouret, J.-B. (2015). Robots that can adapt like animals. *Nature*, 521(7553):503–507.
- de Bruin, K. E., Glette, K., and Ellefsen, K. O. (2025). Generational replacement and learning for high-performing and diverse populations in evolvable robots. In *2025 IEEE Symposium on Computational Intelligence in Artificial Life and Cooperative Intelligent Systems (ALIFE-CIS)*, pages 1–7. IEEE.
- Eiben, A. E., Bredeche, N., Hoogendoorn, M., Stradner, J., Timmis, J., Tyrrell, A., and Winfield, A. (2013). The triangle of life: Evolving robots in real-time and real-space. In *European conference on artificial life (ECAL-2013)*, pages 1–8.
- Fañña, A., Bellas, F., López-Peña, F., and Duro, R. J. (2013). Edhmo: Evolutionary designer of heterogeneous modular robots. *Engineering Applications of Artificial Intelligence*, 26(10):2408–2423.
- Gupta, A., Savarese, S., Ganguli, S., and Fei-Fei, L. (2021). Embodied intelligence via learning and evolution. *Nature Communications*, 12(1):5721.
- Harada, K. and Iba, H. (2024). Lamarckian co-design of soft robots via transfer learning. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 832–840.
- Ijspeert, A. J. (2008). Central pattern generators for locomotion control in animals and robots: a review. *Neural networks*, 21(4):642–653.
- Jelisavcic, M., Glette, K., Haasdijk, E., and Eiben, A. E. (2019). Lamarckian Evolution of Simulated Modular Robots. *Frontiers in Robotics and AI*, 6.
- Lan, G., De Carlo, M., Van Diggelen, F., Tomczak, J. M., Roijers, D. M., and Eiben, A. (2021). Learning directed locomotion in modular robots with evolvable morphologies. *Applied Soft Computing*, 111:107688.
- Le Goff, L. K., Buchanan, E., Hart, E., Eiben, A. E., Li, W., De Carlo, M., Winfield, A. F., Hale, M. F., Woolley, R., Angus, M., et al. (2022). Morpho evolution with learning using a controller archive as an inheritance mechanism. *IEEE Transactions on Cognitive and Developmental Systems*, 15(2):507–517.
- Lehman, J. and Stanley, K. O. (2011). Evolving a diversity of virtual creatures through novelty search and local competition. In *Proceedings of the 13th annual conference on Genetic and evolutionary computation*, pages 211–218.
- Lipson, H. and Pollack, J. B. (2000). Automatic design and manufacture of robotic lifeforms. *Nature*, 406(6799):974–978.
- Luo, J., Miras, K., Tomczak, J., and Eiben, A. E. (2023). Enhancing robot evolution through Lamarckian principles. *Scientific Reports*, 13(1):21109.
- Luo, J., Stuurman, A. C., Tomczak, J. M., Ellers, J., and Eiben, A. E. (2022). The Effects of Learning in Morphologically Evolving Robot Systems. *Frontiers in Robotics and AI*, 9:797393.
- Mertan, A. and Cheney, N. (2024). Towards multi-morphology controllers with diversity and knowledge distillation. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 367–376.
- Miras, K., De Carlo, M., Akhatou, S., and Eiben, A. E. (2020). Evolving-Controllers Versus Learning-Controllers for Morphologically Evolvable Robots. In *Applications of Evolutionary Computation*, volume 12104, pages 86–99. Springer International Publishing.
- Mockus, J. (1974). On bayesian methods for seeking the extremum. In *Proceedings of the IFIP Technical Conference*, pages 400–404.
- Mouret, J.-B. and Clune, J. (2015). Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*.
- Nolfi, S., Bongard, J., Husbands, P., and Floreano, D. (2016). *Evolutionary robotics*. Springer.
- Nordmoen, J., Veenstra, F., Ellefsen, K. O., and Glette, K. (2021). Map-elites enables powerful stepping stones and diversity for modular robotics. *Frontiers in Robotics and AI*, 8:639173.
- Norstein, E. S., Ellefsen, K. O., and Glette, K. (2022). *Open-ended search for environments and adapted agents using MAP-elites*, page 651–666. Springer International Publishing.
- Saito, T. and Oka, M. (2024). Effective design and interpretation in voxel-based soft robotics: A part assembly approach with bayesian optimization. In *Artificial Life Conference Proceedings 36*, page 26. MIT Press.
- Samuelsen, E. and Glette, K. (2014). Some distance measures for morphological diversification in generative evolutionary robotics. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, pages 721–728.
- Sims, K. (1994). Evolving 3d morphology and behavior by competition. *Artificial life*, 1(4):353–372.
- van Diggelen, F., Ferrante, E., and Eiben, A. (2024). Comparing robot controller optimization methods on evolvable morphologies. *Evolutionary Computation*, 32(2):105–124.
- Williams, C. K. and Rasmussen, C. E. (2006). *Gaussian processes for machine learning*, volume 2. MIT press.
- Zhao, A., Xu, J., Konaković-Luković, M., Hughes, J., Spielberg, A., Rus, D., and Matusik, W. (2020). RoboGrammar: graph grammar for terrain-optimized robot design. *ACM Transactions on Graphics*, 39(6):1–16.